

Tema2. Control de flujo

September 23, 2024

1 El lenguaje de programación Python

- Tema 2. Estructuras de control
- Ponente: Ana María Blanco Benito

2 Contenidos

- Estructuras de control
- Estructuras condicionales o de selección
- Estructuras repetitivas o bucles
- Otras instrucciones (break, continue, else, pass)

3 Estructuras condicionales o de selección

- Ejecutan un bloque u otro en función del resultado verdadero o falso de una condición
- Se utiliza la indentación para organizar los bloques
- Para unir condiciones se pueden utilizar los operadores lógicos and, or, not

3.1 Condicional if

- Si se cumple la condición se ejecuta el bloque

```
[7]: edad = int(input("Introduce tu edad: "))  
if edad >= 18:  
    print("Eres mayor de edad")
```

Introduce tu edad: 20

Eres mayor de edad

3.2 Condicional if - else

- Si se cumple la condición se ejecuta el bloque correspondiente al if
- Si no se cumple la condición se ejecuta el bloque correspondiente al else

```
[9]: edad = int(input("Introduce tu edad: "))  
if edad >= 18:  
    print("Eres mayor de edad")  
else:
```

```
print("Eres menor de edad")
```

Introduce tu edad: 6

Eres menor de edad

3.3 Condicional if - elif - else

- elif es una abreviación de “else if”, y es útil para evitar un sangrado excesivo
- Se utiliza para evaluar varias condiciones
- Cuando encuentra una condición verdadera ejecuta el bloque correspondiente y finaliza

```
[11]: # En lugar de esto:
precio = float(input("Introduce el precio: "))
if precio < 50:
    categoria = "Económico"
else:
    if precio < 100:
        categoria = "Moderado"
    else:
        if precio < 150:
            categoria = "Caro"
        else:
            categoria = "Muy caro"
print(f"Este producto es de la categoría: {categoria}")

#Hacemos esto, que es equivalente:
precio = estado = float(input("Introduce el precio: "))
if precio < 50:
    categoria = "Económico"
elif precio < 100:
    categoria = "Moderado"
elif precio < 150:
    categoria = "Caro"
else:
    categoria = "Muy caro"

print(f"Este producto es de la categoría: {categoria}")
```

Introduce el precio: 12.5

Este producto es de la categoría: Económico

Introduce el precio: 12.5

Este producto es de la categoría: Económico

3.4 Condicional match

- A partir de la version 3.10, se ha incorporado la instrucción match
- Equivale al switch de otros lenguajes

- Lo mismo que en los casos anteriores, cuando encuentra una condición verdadera, ejecuta el bloque correspondiente y finaliza

```
[14]: # Estados del protocolo HTTP al acceder a una página web
# _ funciona como un comodín y nunca fracasa la coincidencia
# https://docs.python.org/es/3.12/tutorial/controlflow.html#tut-match
mes = int(input("Introduce el número del día de la semana (1-lunes, 7-domingo):"))
match mes:
    case 1:
        print("Lunes")
    case 2:
        print("Martes")
    case 3:
        print("Miércoles")
    case 4:
        print("Jueves")
    case 5:
        print("Viernes")
    case 6:
        print("Sábado")
    case 7:
        print("Domingo")
    case _:
        print("Error. El número debe estar entre 1 y 7 ")

# Se pueden combinar varios literales en un solo patrón usando /
error = int(input("Introduce el número de error: "))
match estado:
    case 401 | 403 | 404:
        print("Acceso no permitido")
```

Introduce el estado: 4019

Algo va mal en Internet

Introduce el estado: 418

4 Estructuras repetitivas o bucles

- Se utilizan para repetir un bloque de código en función de si se cumple una determinada condición
- Si sabemos el número de veces que se va a repetir -> for
- Si no sabemos el número de veces que se va a repetir -> while
- No hay 'do while' como en otros lenguajes

4.1 Bucle while

- Se ejecuta el bloque mientras se cumpla la condición

- Como en el caso de las condicionales, el bloque debe estar indentado
- Los bucle while se ejecutan 0 o n veces

```
[17]: # Ejemplo de validación
numero = int(input("Introduce un número entre 0 y 10"))
while numero < 0 or numero > 10:
    print("Error. El número debe estar entre 0 y 10")
    numero = int(input("Introduce un número entre 0 y 10"))
print(numero)
```

Introduce un número entre 0 y 10 11

Error. El número debe estar entre 0 y 10

Introduce un número entre 0 y 10 -1

Error. El número debe estar entre 0 y 10

Introduce un número entre 0 y 10 5

5

4.2 Bucle for

- La sentencia for en Python es un poco diferente a la de otros lenguajes como Java
- Itera sobre una secuencia, como cadenas de caracteres, listas, tuplas
- Para iterar sobre una secuencia de números, se debe utilizar la función integrada range(), la cual genera progresiones aritméticas

4.2.1 Range

- range(inicio,fin,salto) Por ejemplo range(4,200, 2) genera números desde el 4 al 199 con saltos de 3
- Se puede omitir el parámetro 'salto'. Por ejemplo, range(3,8) genera números de 3 a 7
- Se puede omitir el parámetro inicio Por ejemplo, range(4) genera números de 0 a 3

```
[7]: # Bucle for en python
frase = input("Introduce una frase: ")
for car in frase:
    print(car, end = "")
print()

# Bucle que itera sobre una secuencia de números
for i in range(4,20,2):
    print(i, end=" ")
print()

for i in range(4):
    print(i, end=" ")
print()
```

Introduce una frase: hola

```
hola
4 6 8 10 12 14 16 18
0123
```

5 Bucles anidados

- Es un bucle dentro de otro bucle
- Por cada iteración del bucle exterior se ejecuta el bucle interior completo

```
[35]: for i in range(1, 5):
      for j in range(1, 11):
          print(f'{j} * {i} = {i*j}')
      print()
```

```
1 * 1 = 1
2 * 1 = 2
3 * 1 = 3
4 * 1 = 4
5 * 1 = 5
6 * 1 = 6
7 * 1 = 7
8 * 1 = 8
9 * 1 = 9
10 * 1 = 10
```

```
1 * 2 = 2
2 * 2 = 4
3 * 2 = 6
4 * 2 = 8
5 * 2 = 10
6 * 2 = 12
7 * 2 = 14
8 * 2 = 16
9 * 2 = 18
10 * 2 = 20
```

```
1 * 3 = 3
2 * 3 = 6
3 * 3 = 9
4 * 3 = 12
5 * 3 = 15
6 * 3 = 18
7 * 3 = 21
8 * 3 = 24
9 * 3 = 27
10 * 3 = 30
```

```
1 * 4 = 4
```

```
2 * 4 = 8
3 * 4 = 12
4 * 4 = 16
5 * 4 = 20
6 * 4 = 24
7 * 4 = 28
8 * 4 = 32
9 * 4 = 36
10 * 4 = 40
```

6 Otras instrucciones: break, continue, else en bucles y pass

- En general no se recomienda el uso de estas estructuras
- La sentencia break termina el bucle for o while más anidado.
- Un bucle for o while puede incluir una cláusula else.
- En un bucle while, se ejecuta después de que la condición del bucle se vuelva falsa.
- En cualquier tipo de bucle, la cláusula else no se ejecuta si el bucle ha finalizado con break.
- La declaración continue, continúa con la siguiente iteración del ciclo
- La instrucción pass no hace nada

```
[10]: # Ejemplo para break y else
for i in range(1, 6):
    print(i)
    if i == 3:
        print("Me salgo del bucle")
        break
else:
    print("El bucle se completó sin interrupciones")

# Ejemplo para continue
for i in range(1, 6):
    if i == 3:
        print("Si i es 3, me salto la iteraciób")
        continue
    print("Iteración:", i)

for i in range(2,20):
    pass
```

```
1
2
3
Me salgo del bucle
Iteración: 1
Iteración: 2
Si i es 3, me salto la iteraciób
Iteración: 4
```

Iteración: 5

7 Bibliografía

- Apuntes de programación de la UC3M para el grado de Ingeniería Informática. Departamento de Informática. Planning and learning group.
- Guía de estilo de Python: <https://peps.python.org/pep-0008/>
- Documentación oficial de Python para la versión python 3.12.1: <https://docs.python.org/3/>